A Sufficiently Generic Rocket Calculator

July 28, 2026

A Sufficiently Generic Rocket Calculator

Problem statement

Program components

Flight plan

Aerodynamics

Propulsion

Nozzle

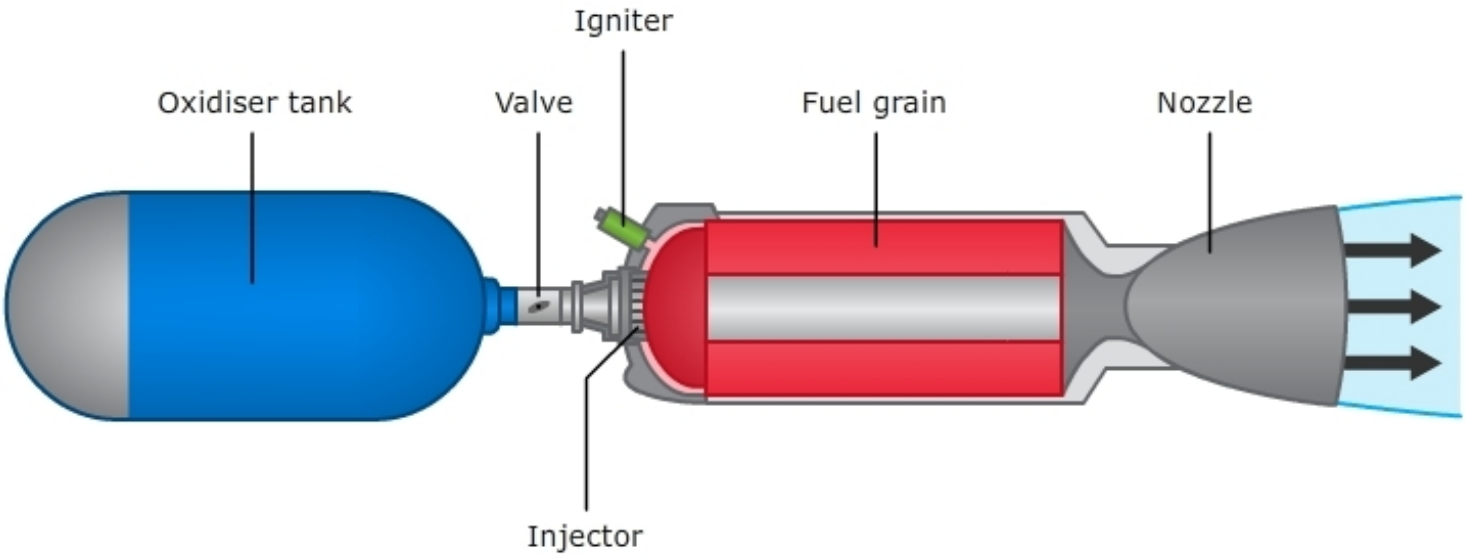
Chemistry with CEA

Outline

Problem statement

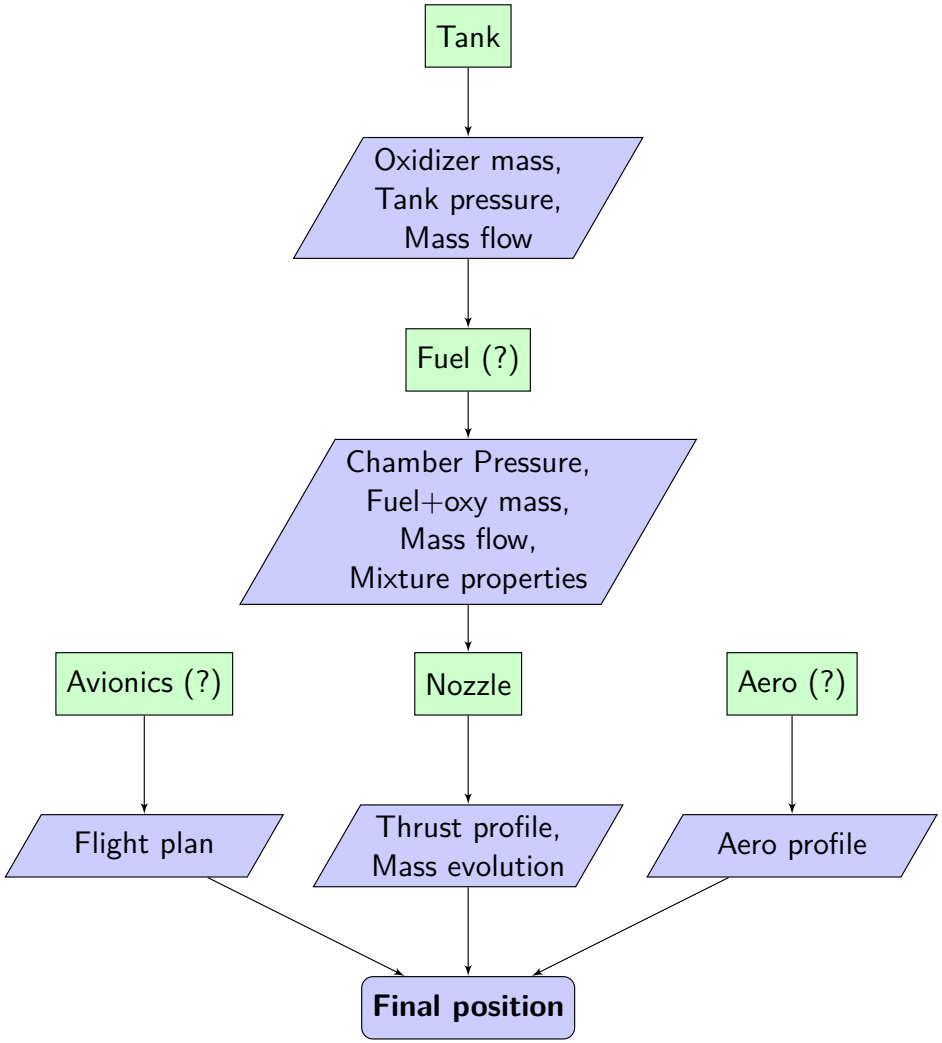
Program components
- Flight plan
- Aerodynamics
- Propulsion
  - Nozzle
  - Chemistry with CEA

Hybrid engine rocket



© Copyright. 2011. University of Waikato. All Rights Reserved.

Data flow


Tank
Oxidizer mass,
Tank pressure,
Mass flow
Fuel (?)
Chamber Pressure,
Fuel+oxy mass,
Mass flow,
Mixture properties
Avionics (?)
Nozzle
Aero (?)
Flight plan
Thrust profile,
Mass evolution
Aero profile
Final position

Attitude

x - position relative to launch site,
v - velocity relative to launch site

Attitude as a lookup table:

$$\mathcal{A} : \mathbb{R}^3, \mathbb{R}^3 \to \mathbb{R}^3 \text{ (or } \mathbb{H}) \qquad (1)$$
$$\mathbf{x}, \mathbf{v} \mapsto (\phi, \theta, \psi)$$

```
Array3D = np.typing.NDArray[np.float64]

def attitude(x: Array3D, v: Array3D) -> Array3D: ...
```


Problem statement
Program components
Flight plan
Aerodynamics
Propulsion
Nozzle
Chemistry with CEA

Aerodynamics

Atmospheric conditions table:

$$\begin{aligned} \mathbf{A} : \mathbb{R}^3 &\to \mathbb{R}, \mathbb{R}, \mathbb{R}, \ldots \\ \mathbf{x} &\mapsto (P_{\text{atm}}, T_{\text{atm}}, w, \ldots) \end{aligned} \tag{2}$$

Drag profile (dimensionless):

$$\begin{aligned} C_{\text{D}} : \mathbb{R}^3, \mathbb{R}^3, \mathbb{R}^3 &\to \mathbb{R} \\ \mathbf{x}, \mathbf{v}, \mathcal{A} &\mapsto C_{\text{D}} \end{aligned} \tag{3}$$

(neglect attitude assuming rocket is aligned with velocity vector)

```
def drag(x: Array3D, v: Array3D) -> np.float64: ...
```

Depends on the shape of the rocket

Nozzle thrust profile

Thrust profile:

$$F_{\mathrm{T,e}} = F_{\mathrm{T,e}}(t) \tag{4}$$

Ideally, the engine thrust depends on the mass flow, exit velocity, exit pressure and nozzle exit cross-section:

$$F_{\mathrm{T,e}}(t) = \dot{m}(t)u_{\mathrm{e}}(t) + P_{\mathrm{e}}(t)A_{\mathrm{e}} \tag{5}$$

Real thrust is then the engine thrust adjusted for atmospheric conditions, efficiency, and drag:

$$\begin{aligned} F_{\mathrm{T}} &= F_{\mathrm{T,e}} \cdot \eta - P_{\mathrm{amb}} \cdot A_{\mathrm{e}} - F_{\mathrm{D}} \\ &= (\dot{m}u_{\mathrm{e}} + P_{\mathrm{e}}A_{\mathrm{e}}) \cdot \eta - P_{\mathrm{amb}}A_{\mathrm{e}} - F_{\mathrm{D}} \end{aligned} \tag{6}$$

η is the efficiency parameter that depends on the nozzle shape

Typical convergent-divergent nozzle

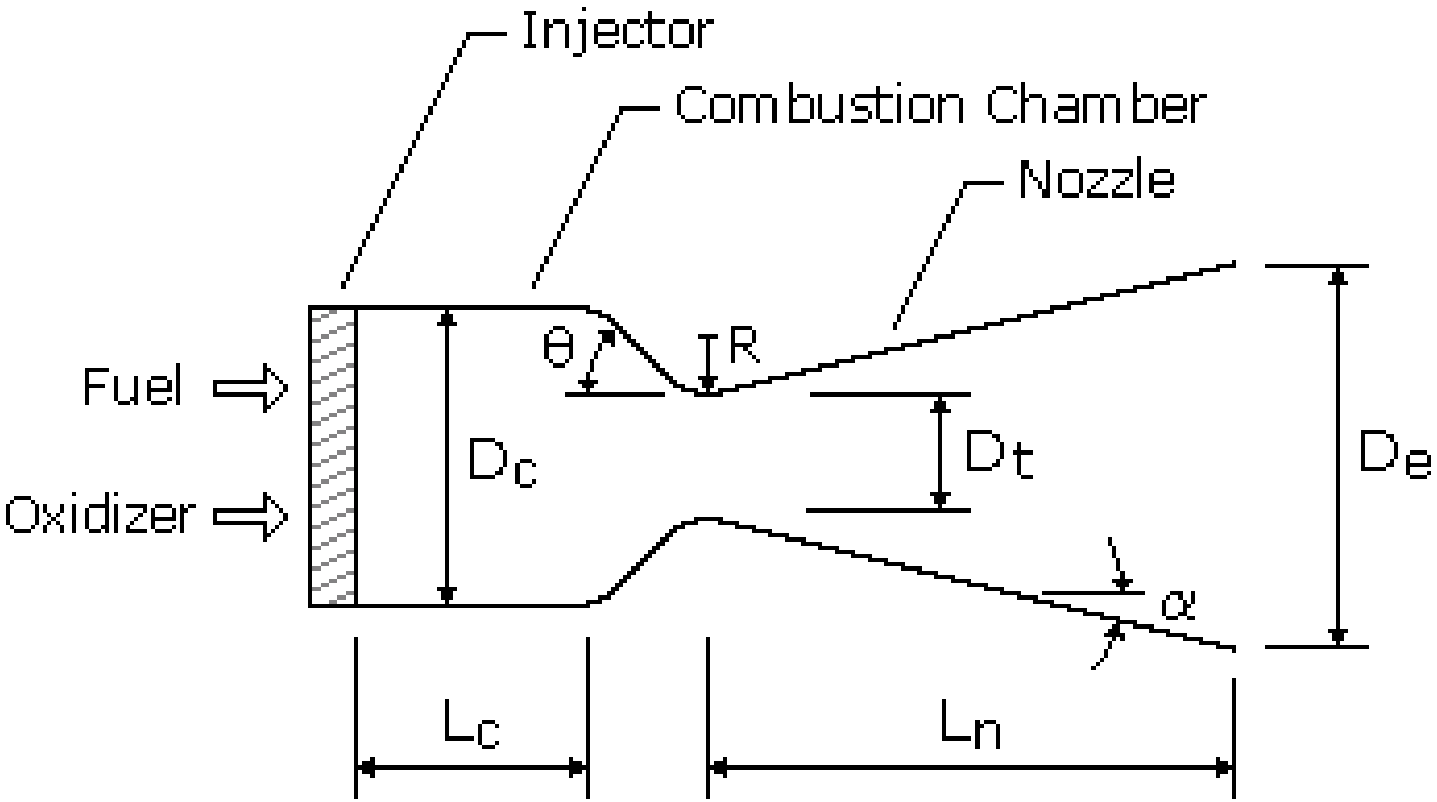
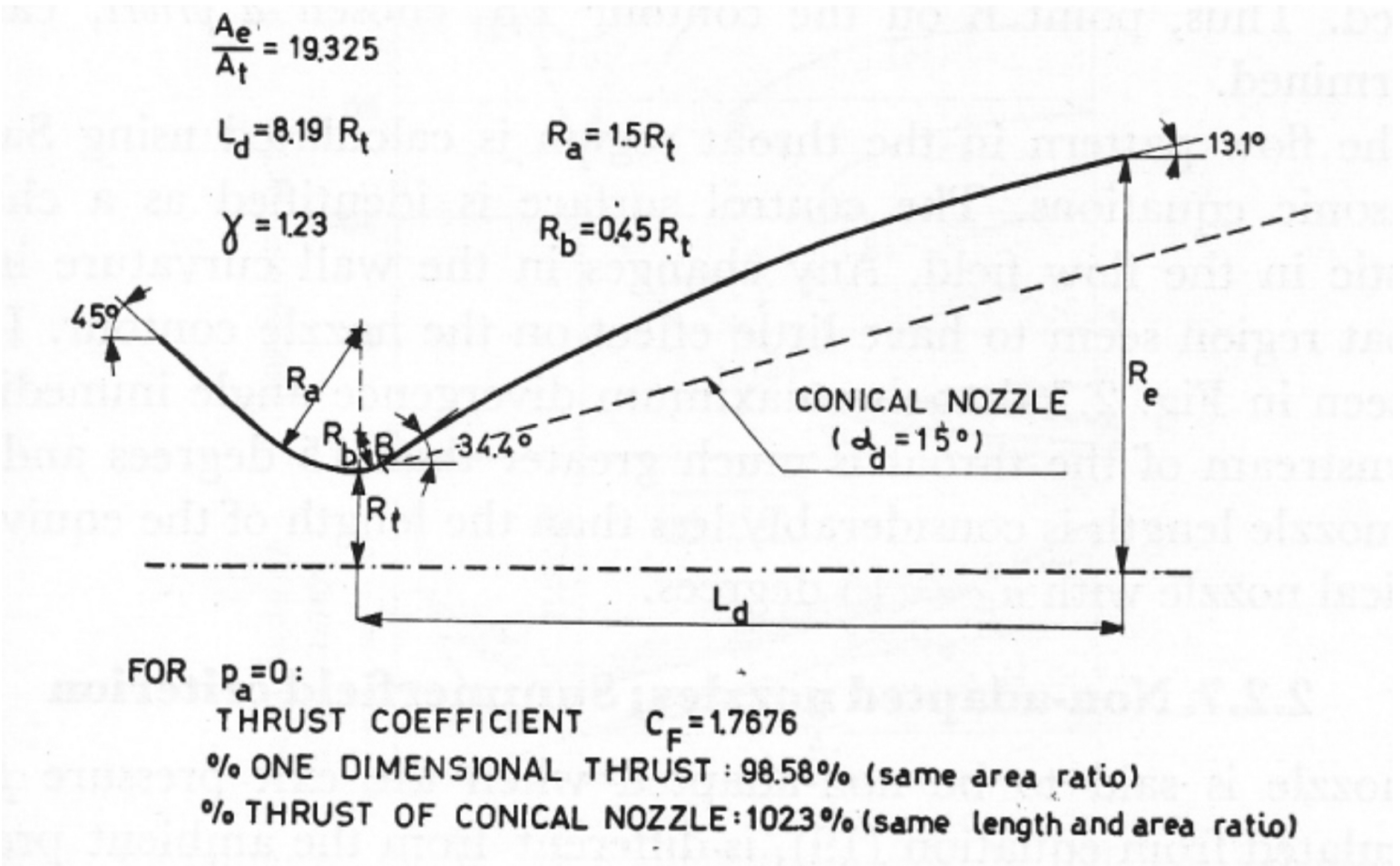


Figure 1.4

Parabolic nozzle



Nozzle efficiency

z - axial position with nozzle entrance ("chamber") at $z = 0$

Then, model nozzle shape as some form-factor function $f(z)$

Nozzle efficiency is a function of this form-factor, $\eta[f(z)]$

Need simulations to determine exactly:

- `NozzleMate`: a Python solution
- further options?

Thrust calculations

$$\dot{m} = \frac{P_c A_t}{c^*}$$

where P_c is the chamber pressure and c^* is the *characteristic velocity* computed with CEA.

Engine thrust:

$$F_{T,e} = \dot{m} I_{sp} g_0 = C_f A_t P$$

compute with I_{sp} or C_f (*thrust coefficient*) provided by CEA.

Thrust profile inputs

Engine thrust:

$$F_{\mathrm{T}}(t) = F_{\mathrm{T}}[\dot{m}(t), u_{\mathrm{e}}(t), P_{\mathrm{e}}(t), A_{\mathrm{e}}] \tag{7}$$

We calculate all of these using the CEA library

Real thrust accounting for efficiency and atmospheric conditions:

$$F_{\mathrm{T}}(t) = F_{\mathrm{T}}[\dot{m}(t), u_{\mathrm{e}}(t), P_{\mathrm{e}}(t), A_{\mathrm{e}}; \eta, \mathbf{A}] \tag{8}$$

CEA solver

```
mixture = [
    {
        name: "H2(L)", temp: 20.27,
        fraction: 1.0, fuel_weight: 1.0, oxidant_weight: 0.0
    },
    {
        name: "O2(L)", temp: 90.17,
        fraction: 1.0, fuel_weight: 0.0, oxidant_weight: 1.0
    }
]
```

Isolating the essential arguments of the CEA rocket solver, one can define a function of the following form:

```
def solve(
    mixture : dict,              # mixture component dictionary
    pc      : np.float64,        # chamber pressure
    subar   : list[np.float64], # subsonic area ratio
    supar   : list[np.float64]  # supersonic area ratio
) -> cea.RocketSolution:

    ... # implementation details

    solver.solve(solution, weights, pc, pi_p, subar=subar, supar=supar, hc)

    return solution.c_star, solution.Cf
```